

```
class Date{
  int d;
  int m;
  int y;
  void displayDate(){ }
}
```

```
class Person{
  string name;
  void displayPerson(){
  }
}
```

```
class Employee:Person // inheritance{
  Date doj; // Associatation
  Date *dot;
  int empid;
  double salary;
  Person p;
}
```

Employee e1;

name	empid	salary	doj	dot
	1	1000	d m y	null

e2

name	empid	salary	doj	dot
			d m y	null

e1.name

e2.name

e1.displayPerson()

e1.doj.d;

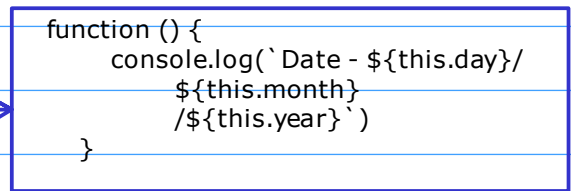
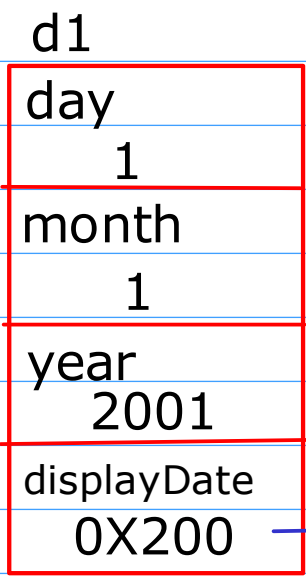
e1.doj.displayDate()

Association

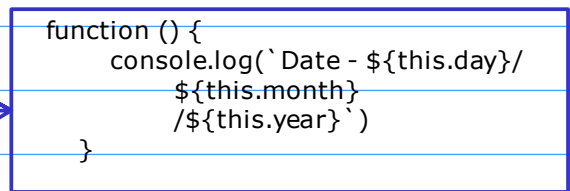
- has-a relationship
- we create an instance of the dependency class inside the dependent class
- Employee has-a DateofJoining
(Dependent) (Dependency)
- used When multiple instances of dependency class is required to be created inside the dependent class

Inheritance

- is-a relationship
- we inherit the parent class inside the child class
- Employee is-a Person
(Child/Derived/sub) is-a (Parent/Base/Super)
- when only single instance of the parent is required inside the child class



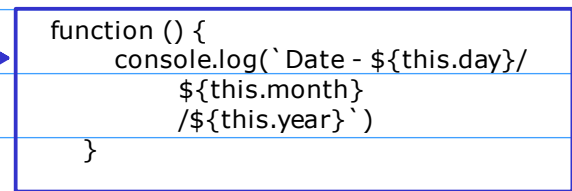
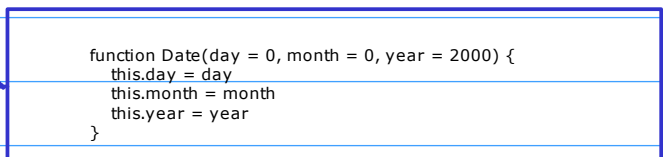
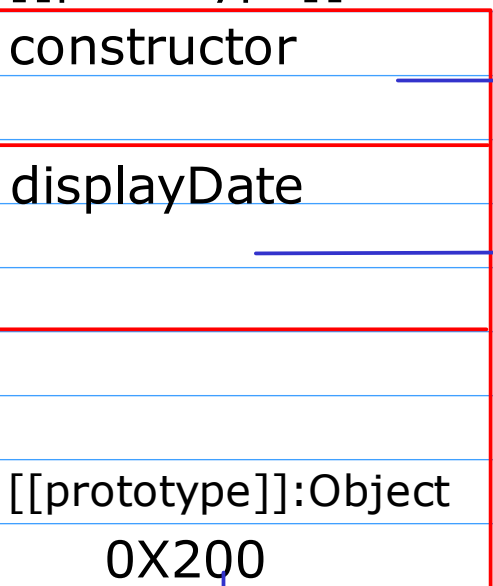
0X200



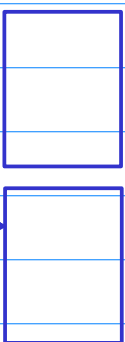
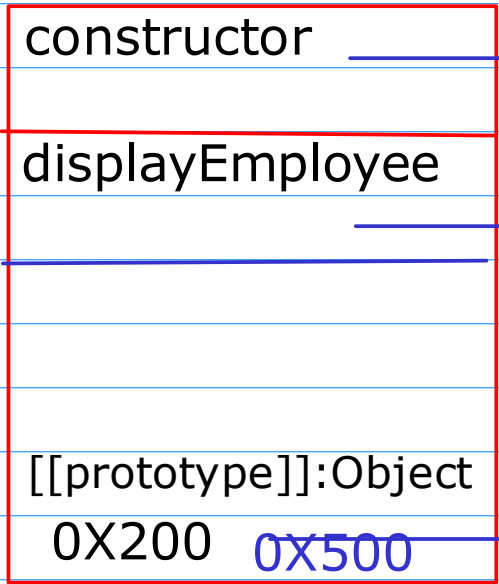
0X300



[[prototype]]:Date

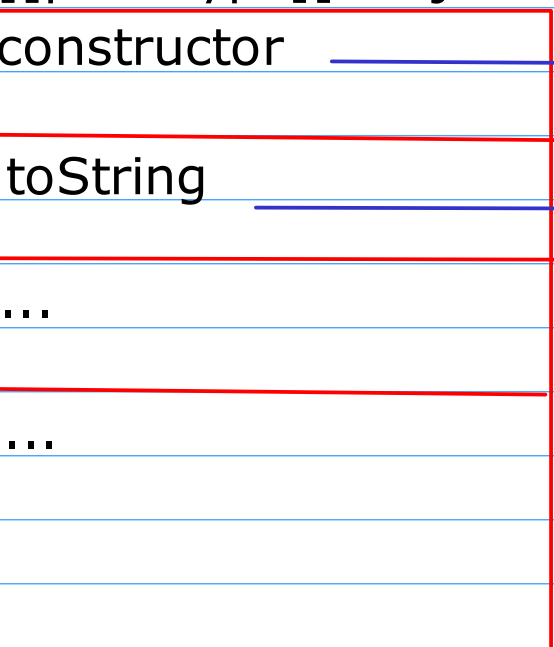


[[prototype]]:Employee

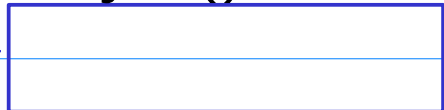


0X300

[[prototype]]:Object



Object()

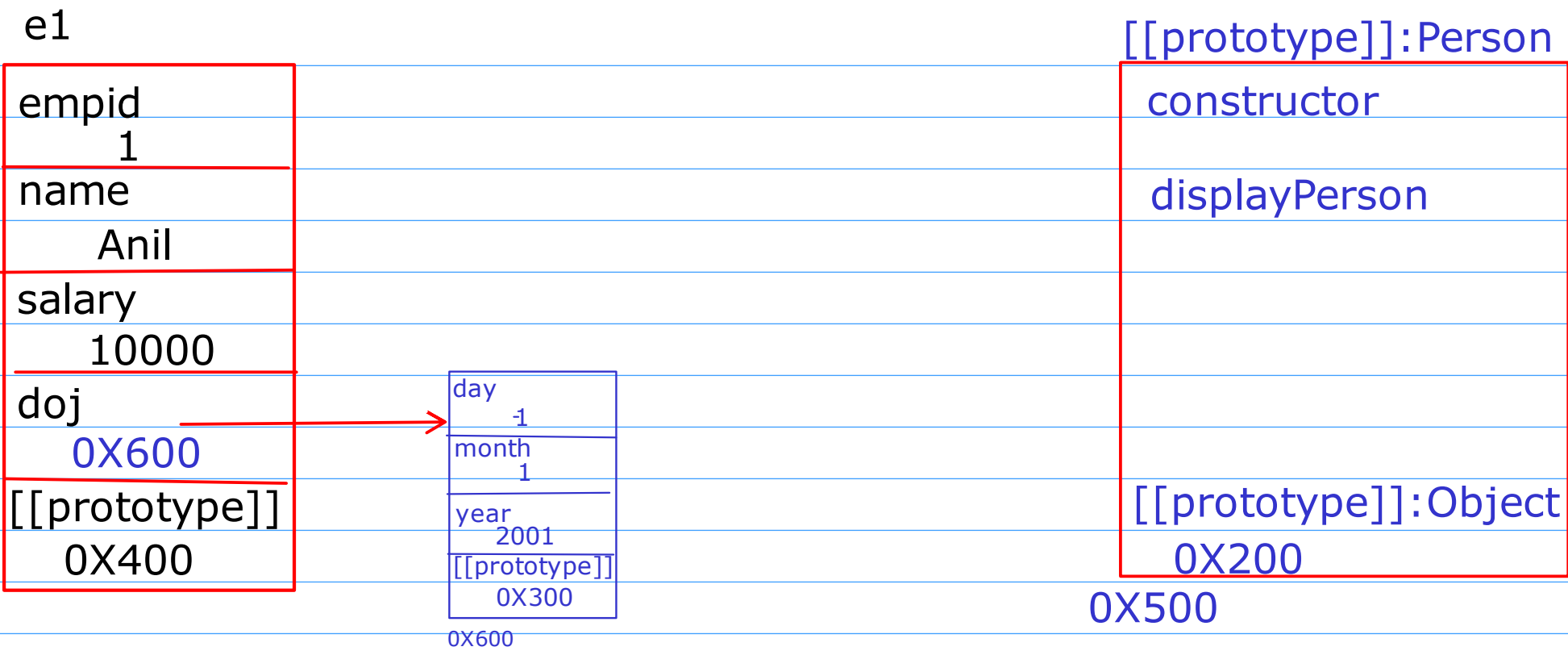


toString()

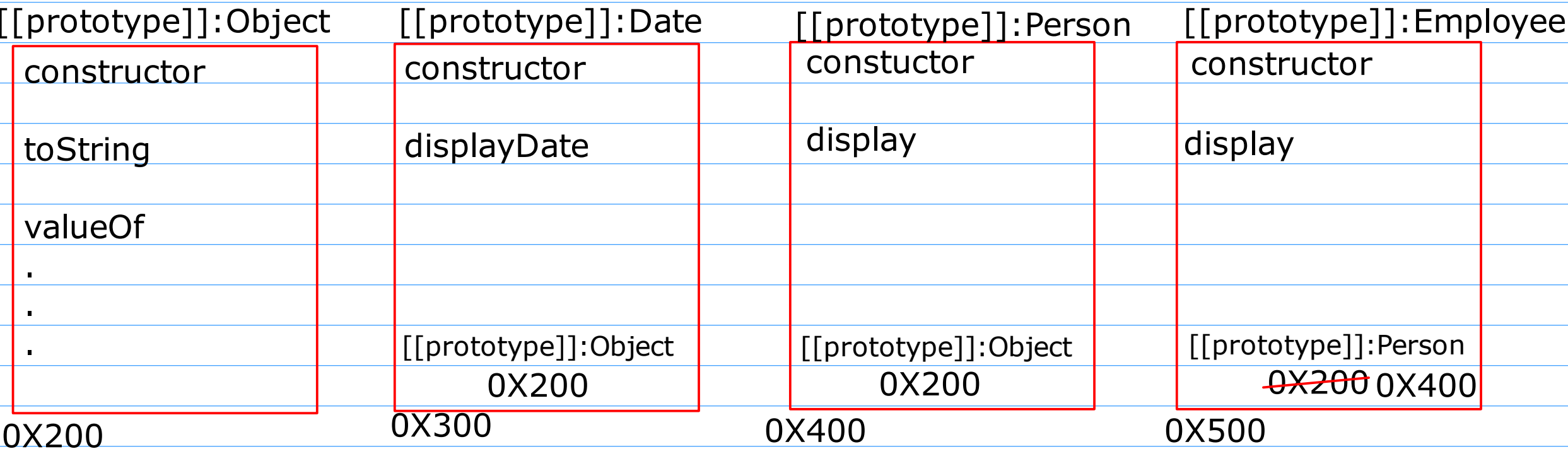
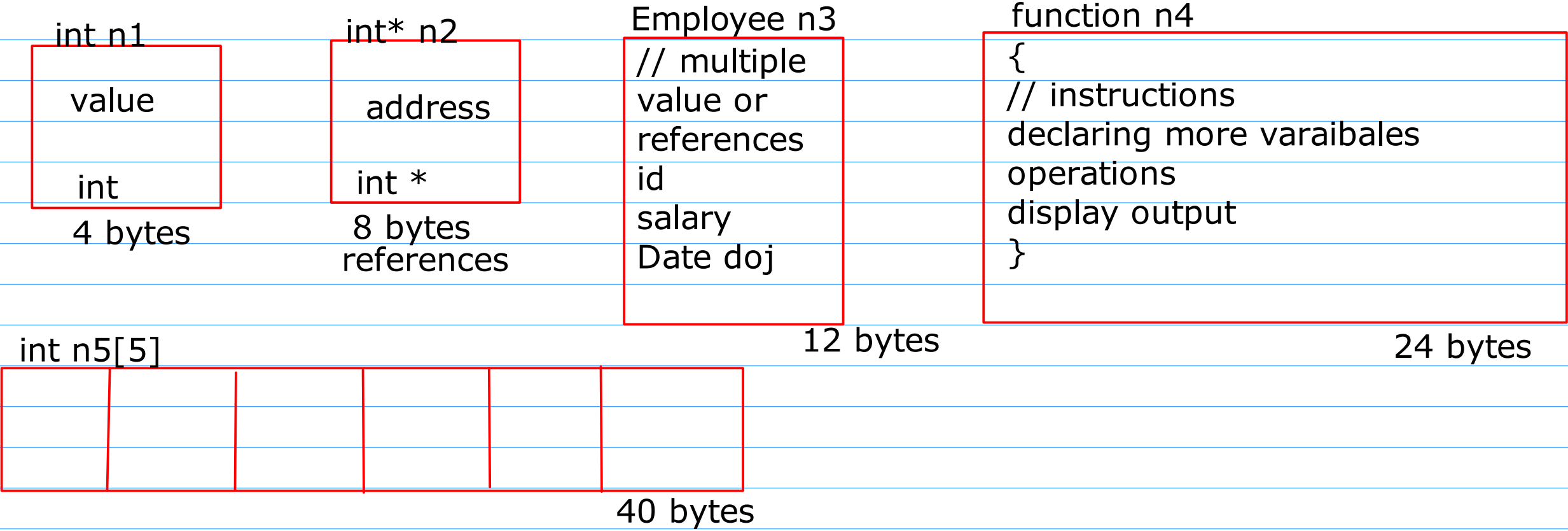


0X400

0X200



Variable



Object.setPrototypeOf(Employee.prototype, Person.prototype)

```
int arr[5]
int *arr = new int[5];
int *arr[5];
int **arr = new int*[5];
```

```
let arr = []

// predicate
function fn(value){
  //checking true or false
}
```

```
arr.push('sunbeam')
arr.push(10)
```

let arr = []

0:'sunbeam',
1:10

[[prototype]]:Object

200 bytes